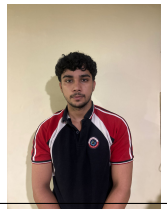
PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Mini OS – A Simulated Operating System using Data Structures & Object-Oriented Programming

Student / Team Information

Team Name: *Team #*	**TEAM PINEAPPLE**
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Singh, Aditi – 2510011501* *Aditi0singh123@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Danu, Shivang – 2510012604* *danushivang794@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Tomar, Ansh – 2510011012* *tomaranjita2@gmail.com* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Zaidi Mehdi, Sameer – 2510370075* *sameerzd110@gmail.com* 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

Operating Systems are a core Computer Science subject, but concepts like process scheduling, memory allocation, file systems, and deadlock handling are usually only studied in theory. Mini OS is a console-based simulation of a real operating system in which every core OS concept is implemented from scratch using an appropriate data structure and Object-Oriented Programming (OOP) design, so the theory is reinforced by working code rather than diagrams alone. The project also strengthens practical DSA and OOP skills that are heavily tested in campus placement drives, by mapping each OS concept (scheduling, memory management, file systems, deadlocks) to a concrete data structure (queues, heaps, linked lists, trees, graphs) and OOP principle (encapsulation, inheritance, polymorphism, abstraction).

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

Real operating systems (Linux, Windows, macOS) implement these concepts at kernel level, which is inaccessible for academic study. Teaching tools like xv6 exist but require systems-programming/C knowledge unsuited to a semester project. Most college mini-OS projects implement only one isolated module (e.g. only a scheduler) procedurally, without an integrated, object-oriented design. Mini OS instead combines ten interconnected components – process management, four schedulers, memory allocation, file system, deadlock detection, I/O management, and a command shell – into one cohesive OOP simulation.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

- **M1 – Design:** finalize the 10 OS components and their DS/OOP mapping.
- **M2 – Process Management:** process creation & PCB using a linked list.
- **M3 – Scheduling:** FCFS, SJF, Round Robin, Priority as subclasses of a common `Scheduler` base class.
- **M4 – Memory Management:** First/Best/Worst Fit allocation on a linked list of blocks.
- **M5 – File System & Deadlock:** tree-based file system; graph-based deadlock detection.
- **M6 – I/O & Shell:** I/O request queue and a command shell.
- **M7 – Integration, Testing & Documentation.**

Project Approach (3 pts)

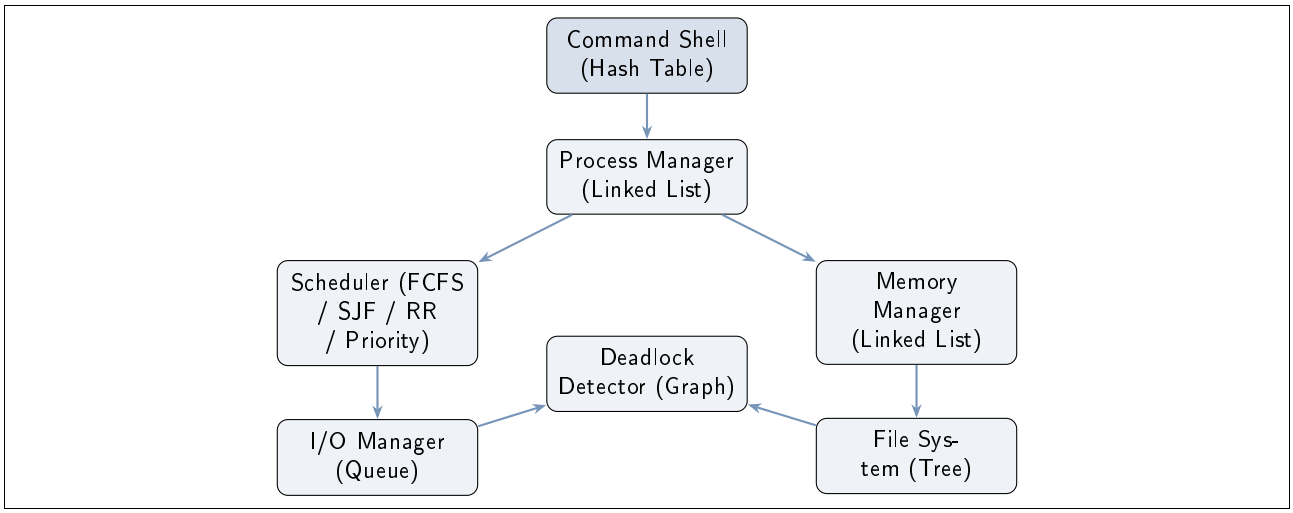
(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

Mini OS is a modular, object-oriented, console-based application. Each module is a class: `Process`, `Scheduler` (base class extended by `FCFS`, `SJF`, `RoundRobin`, `PriorityScheduler` – inheritance & polymorphism), `MemoryManager` (linked list of blocks; First/Best/Worst Fit), `FileSystemNode`

(tree), `DeadlockDetector` (graph, cycle detection), `IOManager` (queue), and `Shell` (hash table of command handlers). Encapsulation hides each component's internal data behind class interfaces; abstraction hides implementation complexity behind simple commands. **Platforms/Technologies:** Language – C++; Core concepts – DSA and OOP; Data Structures – Linked List, Queue, Circular Queue, Priority Queue/Heap, Tree, Graph, Hash Table; Dev Environment – VS Code / any C++ IDE; OS – Windows/Linux; Interface – Console. Modules will be tested individually and then integrated, with sample workloads used to validate scheduling, allocation, and routing logic.

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

A working C++ console prototype of Mini OS delivering: (1) a Process Management module (linked-list PCB); (2) four CPU schedulers – FCFS, SJF, Round Robin, Priority – as polymorphic subclasses; (3) a Memory Allocation module (First/Best/Worst Fit); (4) a tree-based File System (create/delete/read/write/rename/search); (5) a graph-based Deadlock Detector; (6) a queue-based I/O Manager; (7) a Command Shell; and (8) documentation covering design, DS/OOP mapping, and test results.

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

Process attributes (arrival/burst time, priority, memory need) are user-supplied or randomly generated, not read from real hardware. Mini OS runs as a single-user, single-machine console simulation. File system, memory, and I/O devices are simulated entirely in software – no real disk or hardware I/O is performed.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

- C++ Reference – Standard Library Containers: https://en.cppreference.com/w/cpp/container
- GeeksforGeeks – OS Scheduling Algorithms (FCFS, SJF, RR, Priority)
- GeeksforGeeks – Memory Management (First/Best/Worst Fit)
- Silberschatz, Galvin, Gagne – *Operating System Concepts*